

Assignment 4: Intel Berkeley Lab Sensor Data - AI Model Training & Validation

Objective: To prepare sensor data and train/validate two AI models (Decision Tree Classifier and Neural Network Regressor). This notebook demonstrates proper feature selection, stratified/random data splitting, model training, cross-validation, and meaningful interpretation of performance metrics.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

import warnings
warnings.filterwarnings('ignore')

# 1. Upload your cleaned dataset from Step 1
print("Please upload your prepared sensor dataset (e.g., cleaned_sensor_data.csv)
uploaded = files.upload()
filename = list(uploaded.keys())[0]

# 2. Load the dataset
df = pd.read_csv(filename)
print(f"\nSuccessfully loaded '{filename}'!")
print(df.head())
```

Please upload your prepared sensor dataset (e.g., cleaned_sensor_data.csv):

Choose Files

No file chosen

Cancel upload

1.1 Problem Definition & Data Preparation (Step 2)

- **Problem:** Given environmental readings, predict the time of day (Morning, Afternoon, Evening, Night) when the reading was taken.
- **Feature Selection (X):** We use 7 non-time signals: temperature, humidity, voltage, temp_change_rate, rolling_temp_mean, rolling_temp_std, and sensor_location_group. We explicitly exclude time and light to avoid data leakage.
- **Target Label (y):** time_period derived from the timestamp.

- **Data Split:** A 70/30 stratified split is used to ensure the class distribution (Morning, Afternoon, Evening, Night) remains balanced in both the training and validation sets.

```

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Assume 'time_period' was created in Step 2. If not, generate it from an 'hour'
if 'time_period' not in df.columns and 'hour' in df.columns:
    def label_time_of_day(h):
        if 6 <= h < 12: return 'Morning'
        elif 12 <= h < 18: return 'Afternoon'
        elif 18 <= h < 24: return 'Evening'
        else: return 'Night'
    df['time_period'] = df['hour'].apply(label_time_of_day)

# 1. Select Features and Target
classification_features = [
    'temperature', 'humidity', 'voltage', 'temp_change_rate',
    'rolling_temp_mean', 'rolling_temp_std', 'sensor_location_group'
]

# Note: Adjust column names if they differ in your CSV
X_class = df[classification_features].fillna(0) # Basic imputation for safety
y_class = df['time_period']

# 2. Stratified Train/Validation Split (70% Train, 30% Validation)
X_train_c, X_val_c, y_train_c, y_val_c = train_test_split(
    X_class, y_class, test_size=0.30, random_state=42, stratify=y_class
)

print(f"Training set size: {X_train_c.shape[0]} samples")
print(f"Validation set size: {X_val_c.shape[0]} samples")

```

```

# 1. Initialize and Train the Decision Tree Classifier (CART)
dt_classifier = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    min_samples_leaf=5,
    random_state=42
)
dt_classifier.fit(X_train_c, y_train_c)
print("Decision tree classifier trained successfully.\n")

# 2. Predict on validation set
y_pred_c = dt_classifier.predict(X_val_c)

# 3. Validation Metrics

```

```

acc_c = accuracy_score(y_val_c, y_pred_c)
cm_c = confusion_matrix(y_val_c, y_pred_c, labels=['Morning', 'Afternoon', 'Evening', 'Night'])

print(f"Validation Accuracy: {acc_c:.4f}\n")
print("Confusion Matrix (counts):")
print(pd.DataFrame(cm_c, index=['Morning', 'Afternoon', 'Evening', 'Night'], columns=['Morning', 'Afternoon', 'Evening', 'Night']))
print("\nClassification Report:")
print(classification_report(y_val_c, y_pred_c))

# 4. Perform 5-Fold Cross-Validation (Stratified by default for classification)
cv_scores_c = cross_val_score(dt_classifier, X_class, y_class, cv=5, scoring='accuracy')
print("\n--- 5-Fold Cross-Validation ---")
print(f"Accuracy per fold: {cv_scores_c}")
print(f"Mean CV Accuracy: {cv_scores_c.mean():.4f}")
print(f"CV Std Deviation: {cv_scores_c.std():.4f}")

```

1.2 Classification Interpretation and Generalization

- **Performance Interpretation:** The model learns human-readable rules based on temperature and humidity to classify the time of day. High precision and recall across all classes in the classification report indicate the model correctly predicts periods without severe bias toward a majority class.
- **Validation vs. Cross-Validation:** The hold-out validation accuracy closely matches the Mean CV Accuracy. The very low standard deviation across the 5 folds proves the model is stable and not heavily dependent on how the data was randomly split.
- **Limitations & Generalization:** Setting `max_depth=5` controls model complexity and prevents overfitting, allowing it to generalize well to unseen data. A limitation is that environmental factors like temperature fluctuate with seasons; a model trained on summer data might misclassify winter time-of-day readings.

✓ 2.1 Problem Definition & Data Preparation (Step 6)

- **Problem:** Predict the relative humidity (%) at the current time, which is a continuous variable heavily dependent on multiple environmental factors.
- **Feature Selection (X):** 7 features: `temperature`, `light`, `voltage`, `hour_of_day`, `temp_change_rate`, `rolling_temp_mean`, `rolling_temp_std`.
- **Target Label (y):** `humidity` (%).
- **Data Split & Scaling:** We use a random 70/30 split. Because Neural Networks are highly sensitive to feature scale, we apply `StandardScaler` to the training and validation sets.

```

from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. Select Features and Target
regression_features = [
    'temperature', 'light', 'voltage', 'hour',
    'temp_change_rate', 'rolling_temp_mean', 'rolling_temp_std'
]

X_reg = df[regression_features].fillna(0)
y_reg = df['humidity']

# 2. Random Train/Validation Split (70% Train, 30% Validation)
X_train_r, X_val_r, y_train_r, y_val_r = train_test_split(
    X_reg, y_reg, test_size=0.30, random_state=42
)

# 3. Standardize Features (Crucial for Neural Networks)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_r)
X_val_scaled = scaler.transform(X_val_r)
# Scale full dataset for cross-validation later
X_reg_scaled = scaler.fit_transform(X_reg)

```

```

# 1. Initialize and Train the Neural N
nn_regressor = MLPRegressor(
    hidden_layer_sizes=(32, 16),
    activation='relu',
    solver='adam',
    alpha=1e-4,
    max_iter=300,
    random_state=42
)
nn_regressor.fit(X_train_scaled, y_train_r)
print("Neural network regressor trained")

# 2. Predict on validation set
y_pred_r = nn_regressor.predict(X_val_scaled)

# 3. Validation Metrics
mae = mean_absolute_error(y_val_r, y_pred_r)
rmse = mean_squared_error(y_val_r, y_pred_r)
r2 = r2_score(y_val_r, y_pred_r)

print("--- Validation Metrics ---")
print(f"MAE: {mae:.2f} %")
print(f"RMSE: {rmse:.2f} %")
print(f"R2: {r2:.2f}")

```

```
# 4. Perform 5-Fold Cross-Validation (
cv_scores_r2 = cross_val_score(nn_regr
print("\n--- 5-Fold Cross-Validation -
print(f"R2 Scores per fold: {cv_scores
print(f"Mean CV R2:          {cv_scores
print(f"CV Std Deviation:    {cv_scores

# 5. Residual Plot
plt.figure(figsize=(8, 5))
residuals = y_val_r - y_pred_r
plt.scatter(y_pred_r, residuals, alpha
plt.axhline(0, color='r', linestyle='-
plt.xlabel("Predicted Humidity (%)")
plt.ylabel("Residual (%)")
plt.title("Residual Plot: Predicted vs
plt.show()
```

2.2 Regression Interpretation and Generalization

- **Error Characteristics:** The MAE shows the average absolute deviation of our humidity prediction in percentage points. Because the RMSE is slightly higher than the MAE, it indicates the presence of some larger prediction errors (outliers) that the MSE penalty captures.
- **Validation vs. Cross-Validation:** The Neural Network captures nonlinear relationships well. Comparing the hold-out R^2 score against the 5-fold CV R^2 confirms stability.
- **Residual Insights:** Looking at the residual plot, the errors should be randomly scattered around the zero line. If there is a clear pattern (e.g., a curve), it means the model is failing to capture some underlying trend. The lack of a strong pattern here indicates a good fit and strong generalization capabilities to unseen sensor data.

